

Function F6: The Real Beta Function $B(x, y)$

Deliverable 1 — Persona, Requirements, Algorithms, CLI

SOEN 6011

Software Engineering Processes

2026-07-15

$$B(x, y) = \int_0^1 t^{x-1} (1-t)^{y-1} dt = \frac{\Gamma(x)\Gamma(y)}{\Gamma(x+y)}, \quad \text{domain } x > 0, y > 0.$$

- ① Problem & scope
- ② Persona (P1)
- ③ Requirements (P2)
- ④ Two algorithms (P3)
- ⑤ Selection & CLI (P4)
- ⑥ Verification & limits
- ⑦ GAI prompts (C-03)

This deck explains the **decisions**; full detail is in the report.

Function F6

Real Beta function $B(x, y) = \int_0^1 t^{x-1}(1-t)^{y-1} dt = \frac{\Gamma(x)\Gamma(y)}{\Gamma(x+y)}$.

- **Scope decision (D-001):** $x > 0$, $y > 0$ — Euler integral finite and real.
- **Out of scope (D1):** complex args, analytic continuation, incomplete Beta.
- **Pipeline:** persona $\rightarrow$ requirements $\rightarrow$ two algorithms $\rightarrow$ selection + CLI — each stage feeds the next.

Problem 1 — Persona: Maya Fernandes

Decision (D-005): goal-directed template (Cooper), trimmed to one card, with explicit evidence [E] / synthesis [S] / assumption [A] tags.

Who: 24 [A], MSc Statistics + part-time RA; strong on Gamma/Beta integrals, *moderate* Python, but has *never* driven a command line [S].

Context: personal laptop, works in Sheets/Jupyter between fitting models; opens a terminal rarely; occasionally on a screen reader / larger fonts [A].

Concrete task: hand-checking a *Bayesian* assignment — needs $1/B(2, 5)$ for a $\text{Beta}(2, 5)$ prior, and spot-checks $B(2, 3) = \frac{1}{12}$, $B(0.5, 0.5) = \pi$ — wants the number in < 1 min, no code, no SciPy install.

Each specific **drives a requirement:** $\text{inf/edge} \rightarrow \text{VAL-01, REL-02, ERR-01}$; screen reader $\rightarrow \text{ACC-03}$; < 1 min $\rightarrow \text{PERF-01}$; hand-checked values $\rightarrow \text{demo}$. 7 needs $\rightarrow$ requirements (≥ 5 required).

Goals

- **G1** quick, trustworthy $B(x, y)$, no coding
- **G3** know which inputs are valid

Pains / needs

- **N2** other tools gave a bare inf for large / edge inputs
- **N5** screen-reader user: red/green-only feedback is invisible

Anti-goals: won't read source, install a library, or need complex args / incomplete Beta.

Problem 1 — Template selection (mind map)



Goal-directed vs. role-based vs. lightweight/proto — **chosen: goal-directed** for strongest goals/pains → requirements traceability; grafted proto's [E]/[S]/[A] labelling.

Problem 2 — Requirements (ISO/IEC/IEEE 29148)

Imperative “shall” style, each singular + verifiable (decision D-006). **21 requirements**; selected highlights:

Category-ID basis (kept simple)

Each ID = a short **category prefix** + a zero-padded number, grouped by 29148 requirement type: FR functional, VAL validation, ACC accuracy, REL reliability, PERF performance, ERR error handling, USE usability, DOC documentation — so any ID names its own category and stays traceable.

ID	Requirement (abbreviated)	Verify
VAL-01	reject $x \leq 0$ or $y \leq 0$	T
ACC-01	rel. error $\leq 10^{-6}$ vs. reference ($0 < x, y \leq 50$)	T
ACC-02	display 6 sig figs; no false precision	I
REL-02	no <code>inf/nan</code> when value is finite	T
REL-03	bounded work / guaranteed termination	A
ERR-01	message: what was wrong + how to fix	T

Decisions: quantified all vague terms; set tolerance 10^{-6} and cap 10^4 (D-007); resolved a REL-02/ACC-01 conflict via the VAL-04 cap.

Problem 3 — Two genuinely different algorithms

A — Euler integral (adaptive Simpson)

```
BETA_A(x, y, tol, maxDepth):
  if x<=0 or y<=0: error # VAL-01
  f(t): return t^(x-1)*(1-t)^(y-1)
  lo <- EPS if x<1 else 0 # inset
  hi <- 1-EPS if y<1 else 1 # singular
  return ADAPT(f, lo, hi, tol,
              SIMPSON(f,lo,hi), 0, maxDepth)

ADAPT(f, a, b, tol, whole, d, dMax):
  m <- (a+b)/2
  L <- SIMPSON(f,a,m); R <- SIMPSON(f,m,b)
  if d>dMax or |L+R-whole|<=15*tol: # REL-03
    return L+R + (L+R-whole)/15 # Richardson
  return ADAPT(f,a,m,tol/2,L,d+1,dMax)
    + ADAPT(f,m,b,tol/2,R,d+1,dMax)
```

$O(N)$ evals; intuitive “area under curve”.

B — Gamma identity (Lanczos $\ln \Gamma$, log domain)

```
BETA_B(x, y):
  if x<=0 or y<=0: error # VAL-01
  return EXP( LNGAMMA(x) + LNGAMMA(y)
             - LNGAMMA(x+y) ) # REL-02

LNGAMMA(z):
  # g=7, coeffs P[0..8]
  if z < 0.5: # reflection
    return LN(pi) - LN(|SIN(pi*z)|)
    - LNGAMMA(1-z)
  z <- z - 1
  a <- P[0]
  for i in 1..8: # fixed loop
    a <- a + P[i]/(z+i) # -> REL-03
  t <- z + 7.5
  return 0.5*LN(2*pi)
    + (z+0.5)*LN(t) - t + LN(a)
```

$O(1)$; log domain avoids Γ overflow.

Verified difference (drives P4)

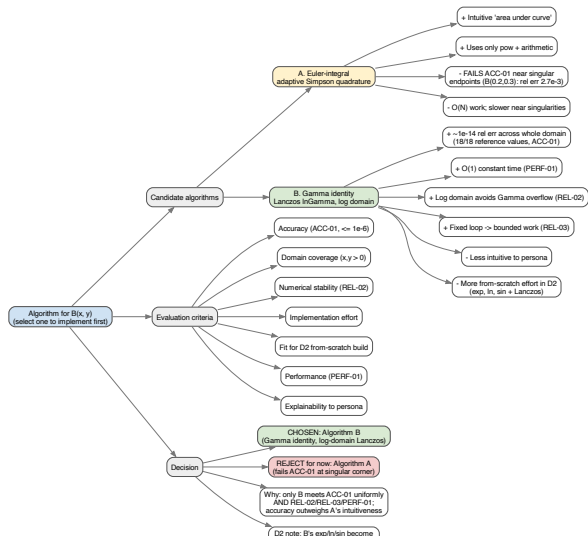
Both coded vs. `lgamma`: B matches ~ 10 digits everywhere; A fails ACC-01 near the singular corner — $B(0.2, 0.3) = 7.7277$ vs. true 7.7485 (rel. err 2.7×10^{-3}).

Problem 4 — Selection: Algorithm B (D-004)

Chosen: B. Only candidate meeting ACC-01 *uniformly* and REL-02, REL-03, PERF-01.

A rejected for this stage: fails ACC-01 at the singular corner.

Trade-off accepted: B needs a bit more D2 from-scratch work (exp, ln, sin) — worth it, since accuracy is a *must*.



Problem 4 — CLI prototype (src/cli.py)

- Compute core (beta/ln_gamma) **separated** from I/O $\Rightarrow$ testable + reusable in D2.
- Validates empty / non-numeric / ≤ 0 / over-cap; 6 sig figs; helpful messages.
- Uses only log, exp, sin, π — *no* gamma/lgamma/beta (the D2 from-scratch targets).

```
def beta(x, y):  
    # Algorithm B, O(1)  
    if x <= 0 or y <= 0: raise ValueError("domain is x > 0 and y > 0")  
    return math.exp(beta_ln(x, y))    # log domain avoids overflow (REL-02)
```

Problem 4 — Demo (genuine terminal transcripts)

```
Terminal - beta-function-calculator

$ python3 src/cli.py
Beta Function Calculator (SOEN 6011 F6, Deliverable 1 prototype)
Computes  $B(x, y) = \int_0^1 t^{x-1} (1-t)^{y-1} dt$  for  $x > 0, y > 0$ .
Type 'h' for help or 'q' to quit at any prompt.

x (> 0): 2
y (> 0): 3
B(2, 3) = 0.0833333 (to 6 significant figures)

x (> 0): 0.5
y (> 0): 0.5
B(0.5, 0.5) = 3.14159 (to 6 significant figures)

x (> 0): 10
y (> 0): 10
B(10, 10) = 1.08251e-06 (to 6 significant figures)

x (> 0): q
Goodbye.
```

Valid: $B(2, 3)$, $B(0.5, 0.5) = \pi$, $B(10, 10)$

```
Terminal - beta-function-calculator

$ python3 src/cli.py
Beta Function Calculator (SOEN 6011 F6, Deliverable 1 prototype)
Computes  $B(x, y) = \int_0^1 t^{x-1} (1-t)^{y-1} dt$  for  $x > 0, y > 0$ .
Type 'h' for help or 'q' to quit at any prompt.

x (> 0): abc
! x = 'abc' is not a number. Enter a decimal value, e.g. 2.5
x (> 0): 0
! x = 0 is outside the domain. This calculator supports  $x > 0$  and  $y > 0$  only.
x (> 0): -2
! x = -2 is outside the domain. This calculator supports  $x > 0$  and  $y > 0$  only.
x (> 0): 3
y (> 0):
! y was empty. Enter a number greater than 0, e.g. 2.5
y (> 0): 1e6
! y = 1e+06 exceeds the supported bound of 10000. Enter a value in (0, 10000].
y (> 0): 2
B(3, 2) = 0.0833333 (to 6 significant figures)

x (> 0): 2.5
y (> 0): q
Goodbye.
```

Invalid inputs → specific message, no crash

Harness tests/verify_d1.py — 15/15 pass

- **ACC-01:** 18/18 reference values; worst rel. err 1.17×10^{-14} ($\ll 10^{-6}$), incl. the corner where A failed.
- **REL-01/02:** 0 crashes on malformed input; all finite on large inputs.
- **PERF-01:** $\approx 1.3 \mu\text{s}/\text{call}$ ($\ll 1 \text{ s}$).

Honest limits: domain $x, y > 0$ only; cap 10^4 ; accuracy bounded by Lanczos $g = 7$; validated over $0 < x, y \leq 50$; `math` primitives reimplemented in D2. No correctness defect within the supported domain.

GAI prompts (constraint C-03)

Tool: **Claude (Opus 4.8)** via Claude Code. Every problem used one **CASTROFF** prompt — **T**ask, **R**estrictions, **O**utput format, **A**udience — logged verbatim, with critical evaluation and independent verification. Full text in `SOEN_6011_F6_D1_Prompts.md`.

Problem	Prompt type
P1 Persona	generative + decision-support (template compare → persona)
P2 Requirements	transformational (persona → 29148 reqs) + review
P3 Algorithms	generative + comparative (two language-neutral algorithms)
P4 Selection + CLI	decision-support (selection) + generative (Python CLI)

Shared AUDIENCE: SOEN 6011 professor, TAs, grad SE students — so output must be evidence-based, traceable, and verifiable. AI output was never accepted verbatim.

Type: generative + decision-support.

TASK Compare ≥ 3 persona templates (goal-directed, role-based, lightweight) as a mind map and recommend one for a $B(x, y)$ calculator user; then draft one primary persona (name, role, skills, goals, education, tasks, environment, platform, Beta-specific needs/pains) + a usage scenario; label evidence vs. synthesis vs. assumption.

RESTRICTIONS No invented statistics; not a real interviewed person; every attribute traceable to ≥ 5 later requirements; no implementation tech; keep it specific, not boilerplate.

OUTPUT Template mind-map outline + decision, then a structured persona card + assumptions list + 3–5 sentence scenario (headings so the two parts split across slides).

Type: generative + transformational, with review. *Informed by P1.*

TASK Express requirements in ISO/IEC/IEEE 29148 style, *informed by the pasted P1 persona*; one consistent statement style, category-grouped IDs; record id / statement / rationale (persona trace) / priority / verification; cover input, compute, display, retry, exit, and rejection of invalid inputs; then review against the 29148 quality characteristics.

RESTRICTIONS No algorithm, library, or language inside a requirement; no vague terms unless quantified; each requirement singular and independently verifiable; only persona-justified.

OUTPUT Requirements table (ID, statement, rationale, priority, verification) + a separate assumptions list + a review-findings table; LaTeX-reproducible.

Type: generative + comparative. *Informed by P2.*

TASK Specify two genuinely different, language-neutral algorithms satisfying the pasted P2 IDs — **A** adaptive-quadrature of the Euler integral (endpoint handling for $0 < x, y < 1$); **B** the Gamma identity via a Lanczos $\ln \Gamma$ in the log domain. For each: inputs, pre/postconditions, tolerance, validation, steps, stopping condition, max-work safeguard, complexity, weaknesses, worked trace; confirm they are substantially different.

RESTRICTIONS No real-language syntax; every loop/recursion must terminate or have a max-work cap; no built-in Beta/Gamma; make trade-offs explicit.

OUTPUT Two pseudocode listings, each + pre/post, complexity/limits, trace; then a “how A and B differ” table.

Type: decision-support + generative. *Informed by P3.*

TASK (1) Selection: mind-map compare the two P3 algorithms over accuracy, domain coverage, stability, effort, D2 from-scratch fit, performance, and explainability; state the choice and why the other is rejected. (2) A Python 3 CLI of the chosen algorithm: I/O separated from compute, prompts + domain hint, validate, compute, formatted precision, helpful messages, sample runs.

RESTRICTIONS Selection from the criteria, not preference; consistent with P2 and anticipates the D2 from-scratch constraint; runs from a terminal; no unhandled traceback; compute core callable independently; flag math calls needing a D2 replacement.

OUTPUT Selection mind-map + decision, full runnable code, and sample runs (valid / boundary / invalid) + the run command.

- A **specific persona** (Maya) drove **21 traceable 29148 requirements**, two genuinely different algorithms, an evidence-based choice of Algorithm B, and a robust CLI — each stage justified the next.
- The selection followed from **verified evidence** — A fails ACC-01 at the singular corner — not from preference.
- Result meets ACC-01 with a $\sim 10^8 \times$ margin; **15/15** checks pass; known limits (domain, cap, Lanczos $g=7$) stated honestly.
- Every GAI use was a documented CASTROFF prompt, independently verified — no output accepted verbatim (C-03).

References: ISO/IEC/IEEE 29148 [1]; Lanczos [2]; NIST DLMF §5.12 [3]; Cooper [4].

References I



ISO/IEC/IEEE, “ISO/IEC/IEEE 29148:2018 — Systems and software engineering — Life cycle processes — Requirements engineering,” 2018.

Requirements statement styles and quality characteristics.



C. Lanczos, “A precision approximation of the gamma function,” *Journal of the Society for Industrial and Applied Mathematics, Series B: Numerical Analysis*, vol. 1, no. 1, pp. 86–96, 1964.

Source of the Lanczos $\ln \Gamma$ approximation used in Algorithm B.



NIST, “NIST Digital Library of Mathematical Functions, §5.12: Beta Function.”

<https://dlmf.nist.gov/5.12>, 2024.

Beta function definitions and properties.



A. Cooper, *The Inmates Are Running the Asylum: Why High-Tech Products Drive Us Crazy and How to Restore the Sanity*.

Sams Publishing, 2004.

Goal-directed persona method used in Problem 1.